TUNKU ABDUL RAHMAN UNIVERSITY OF MANAGEMENT AND TECHNOLOGY

FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

ACADEMIC YEAR 2026/2027

AUGUST/SEPTEMBER FINAL ASSESSMENT

BMSE3004 COLLABORATIVE DEVELOPMENT

Final Assessment Question Release Date:	**MONDAY, 17 AUGUST 2026**
Date of Submission:	**FRIDAY, 18 SEPTEMBER 2026**

STUDENT'S DECLARATION OF ORIGINALITY

- By submitting the final assessment answer, I declare that this submitted work is free from all forms of plagiarism and, for all intents and purposes, is my own properly derived work.
- I confirm that the submitted work presents ideas or arguments not derived from any generative AI tools or other sources.
- I agree that using generative AI tools without acknowledgement in developing the final assessment deliverables will be treated as academic misconduct and subject to disciplinary action by the University.

Student Name	Ng Jing Yong
Student ID	25WMR10019
Programme	Bachelor Of Software Engineering

AI USAGE DISCLOSURE FORM

STUDENT :

NAME (BLOCK LETTERS)	STUDENT ID.
1. NG JING YONG	**25WMR10019**

PROGRAMME : **RSW**
COURSE : **BMSE3004 Collaborative**
PROJECT TITLE : **xxxxxxxx**

1. **AI Tool(s) Used (Check that all that apply)**

[ ] None (I/We did not use any AI tools for this assignment)
[/] ChatGPT (Version: 5.4) [ ] Deepseek
[ ] Gemini [ ] Other: _____

2. **Nature of Assistance (Check all that apply)**

[ ] **Brainstorming:** Generating ideas, topics, or outlines.
[ ] **Research:** Summarising long articles or finding concepts.
[/] **Editing:** Checking grammar, spelling, or sentence structure.
[/] **Coding/Math:** Debugging code or explaining a formula.
[ ] **Creation:** Generating images, data, or draft text

3. **Disclosure Statement**

Provide a brief explanation of how AI tools were utilised in completing the task and describe the measures taken to ensure that the final submission reflects your own independent work

<write your statement here>

4. **Verification & Integrity**

[/] I/We have reviewed and verified the accuracy of all facts, data, and citations generated with the assistance of AI tools.
[/] I/We have properly acknowledged and cited any AI-generated text, ideas, or content in accordance with my instructor's requirements.
[/] I confirm that the reasoning, interpretation, and analysis presented in the final submission are entirely my own

Student Signature: NJY **Date: 8/9/2026**

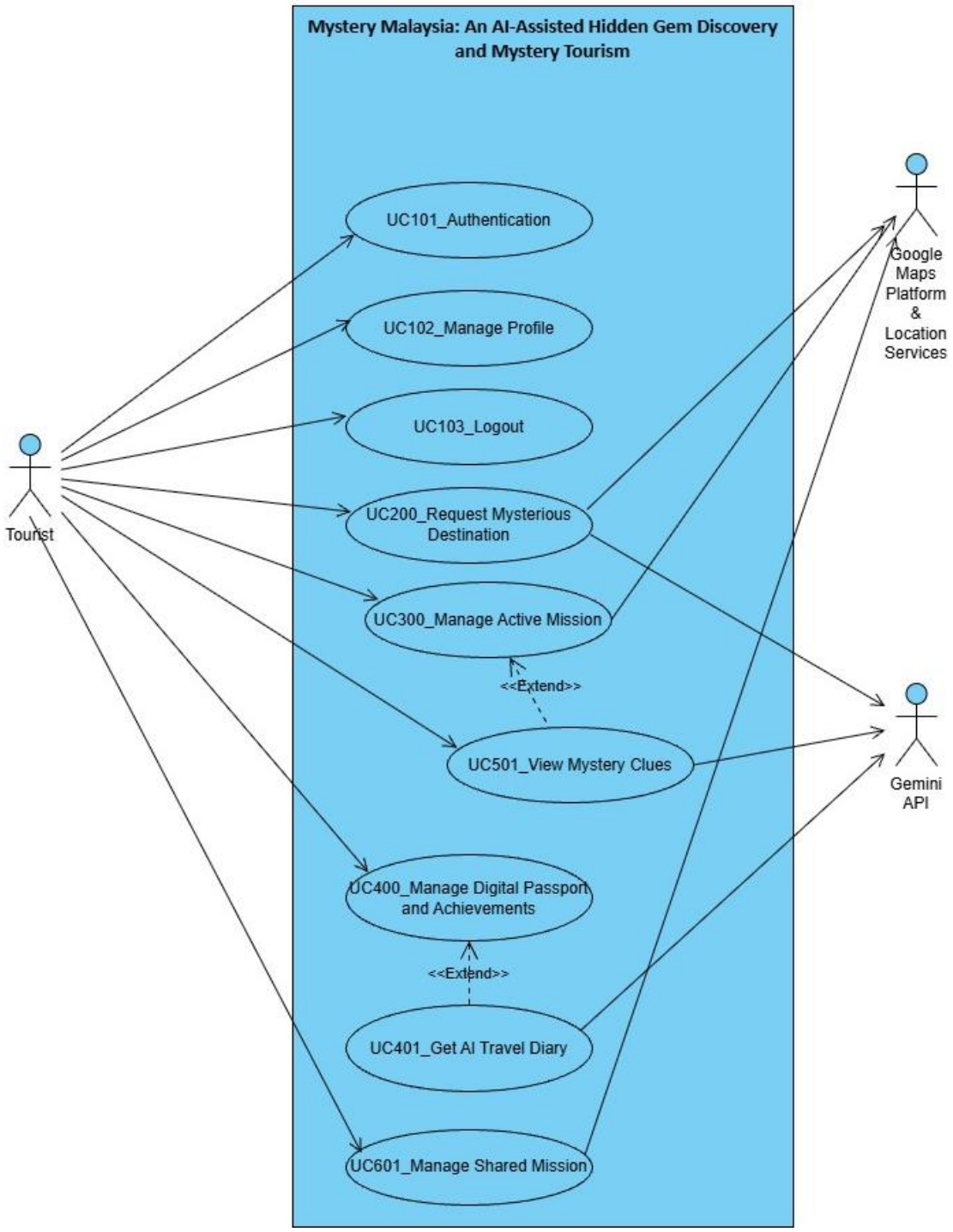
AMM 5.3.2026/SMC 1.4.2026

SECTION A: INTRODUCTION

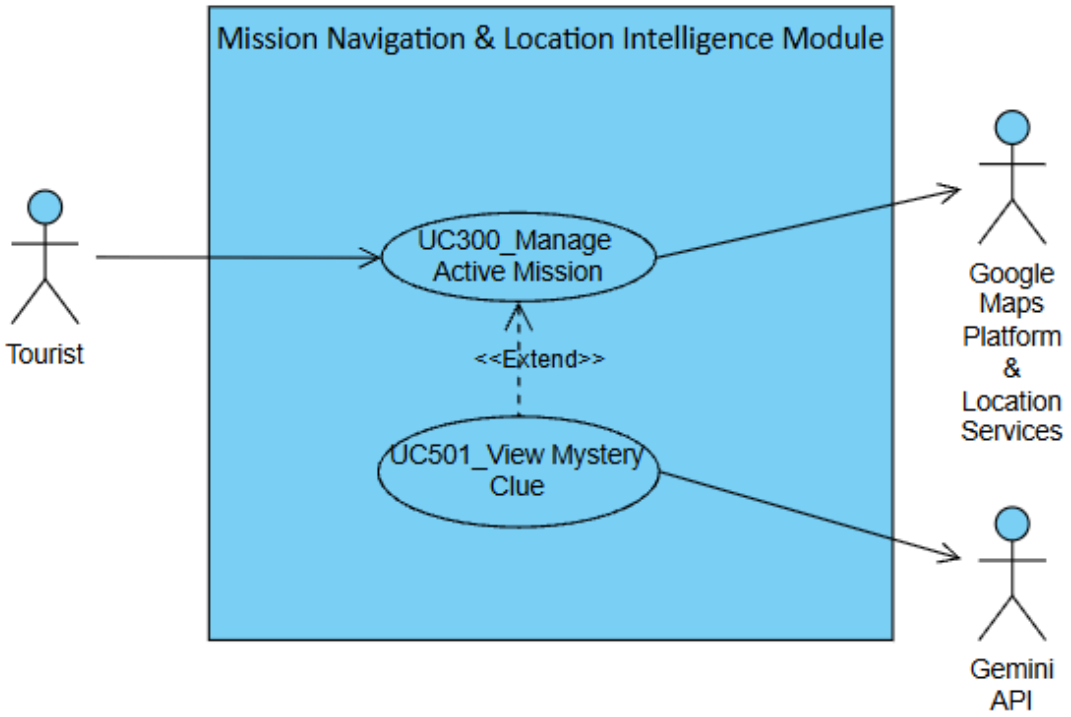
Application / Software Name	Mystery Malaysia
Module Assigned	Mission Navigation & Location Intelligence Module
Module Description	The Mission Navigation & Location Intelligence (active_mission) module manages a tourist's mission from the Ready state through starting, live navigation, pause/resume, arrival verification, completion, or abandonment. While a mission is active, it samples the device's location at controlled intervals and requests server-side Hot and Cold proximity feedback without exposing the hidden destination coordinates in the user interface. It synchronises the latest mission state, handles GPS and network errors, prevents duplicate arrival checks, and updates the stage, status, clue reference, and destination guide after successful geofence verification. The Flutter view accesses this workflow only through the IActiveMissionFacade boundary; clue and hint content remains the responsibility of the separate Clue and Hint module.

SECTION B: REQUIREMENT STUDY

B1(a) Group Use Case Diagram



B1(b) Individual Use Case Diagram



Note: I in charged in UC300_Manage Active Mission only, UC501_View Mystery Clue is handled by another team member (Low Chee Guan)

B2. Use Case Description

Use Case Name	UC300_Manage Active Mission
Objectives	Allows a tourist to manage the complete lifecycle of a mystery mission, including selecting and starting a generated mission, viewing a mystery clue, navigating towards the hidden destination using GPS-based Hot & Cold proximity feedback, pausing or resuming the mission, abandoning the mission, verifying arrival through geofencing, and viewing the destination information after successful verification.
Actor / Activator	Tourist
Pre-Conditions	1. The tourist has successfully logged into the system. 2. A mystery mission has been generated by the AI Hidden Gem Recommendation Engine. 3. The selected mission has not been completed or abandoned. 4. The selected mission is associated with a valid hidden destination and mystery clue. 5. The tourist's device supports GPS and location services. 6. The tourist's device has an active internet connection. 7. The tourist has granted the required location permission. 8. GPS or device location services are enabled.
BASIC FLOW SECTION	
Actor	**System**

	1. System displays generated or paused mission. **[FR300.1] [C1: Display Mission Information]**
2. Tourist selects a mystery mission. **[FR300.1] [A3: Resume Paused Mission]**	3. System displays the mission summary and the **Start Mission** option. **[C1: Display Mission Information]**
4. Tourist selects **Start Mission**. **[FR300.2]**	5. System retrieves the hidden destination ID and geographical coordinates associated with the selected mission without revealing the destination identity. **[FR300.4] [FR300.20] [A1: Mission Data Cannot Be Retrieved] [C4: Destination Concealment]** 6. System verifies that a mystery clue is associated with the selected mission. **[FR300.5]** 7. System creates an active mission session, changes the mission status to **Active**, and records the mission start date and time. **[FR300.3]** 8. System saves the initial mission status and progress to the system database. **[FR300.6]** 9. System begins retrieving the tourist's current latitude, longitude, and GPS accuracy through Google Maps and Location Services. **[FR300.7]** 10. System displays the active mission interface. **[FR300.8] [C2: Display Active Mission Information]**
11. Tourist begins travelling towards the hidden destination.	12. System continuously retrieves the tourist's latest latitude, longitude, and GPS accuracy. **[FR300.7]** 13. System calculates the geographical distance between the tourist's current location and the hidden destination. **[FR300.9]** 14. System compares the current distance with the previously calculated distance to determine whether the tourist is moving closer to or further from the destination. **[FR300.10]** 15. System displays the appropriate Hot & Cold proximity feedback. **[FR300.11] [C3: Hot & Cold Proximity Levels]** 16. System saves the latest active mission status and progress to the system database. **[FR300.6]**

17. Tourist continues travelling based on the Hot & Cold proximity feedback.	18. System continues updating the tourist's location, distance, movement direction, and Hot & Cold proximity feedback. **[FR300.7] [FR300.9] [FR300.10] [FR300.11]** 19. System checks whether the tourist has entered the predefined destination geofence. **[FR300.13] [A5: Tourist Outside Geofence]** 20. System validates the accuracy of the tourist's current GPS location. **[FR300.14] [A6: Insufficient GPS Accuracy]** 21. System verifies the destination visit after confirming that the tourist is within the required geofence radius and the GPS accuracy is acceptable. **[FR300.15] [C5: Destination Verification Rule]** 22. System records the verified visit, including tourist ID, mission ID, destination ID, verified latitude, verified longitude, arrival date, arrival time, and GPS accuracy. **[FR300.16]** 23. System updates and saves the tourist's mission progress. **[FR300.17] [FR300.6]** 24. System retrieves the verified destination information from the database using the destination ID associated with the mission. **[FR300.18] [A7: Destination Information Cannot Be Retrieved]** 25. System displays the destination name, category, description, address, images, and available historical or cultural information. **[FR300.19] [C6: Display Revealed Destination Information]** 26. System changes the mission status to **Completed**. **[FR300.21]** 27. System stops active GPS location tracking. **[FR300.22]**
28. At any time while the mission is active, the tourist can pause the mission. **[A2: Pause Mission]** 29. At any time while the mission is active or paused and before completion, the tourist can abandon the mission. [**A4: Abandon Mission]**	30. At any time while the mission is active, if the tourist selects **View Mystery Clue**, the system activates **UC501_View Mystery Clue**. **[FR300.12]** 31. Use case ends.
ALTERNATE FLOW SECTION	
A1: Mission Data Cannot Be Retrieved	
Occurs at Basic Flow Step 5	

A1.1. System fails to retrieve the hidden destination information or verify that a mystery clue is associated with the selected mission.
A1.2. System displays the Mission Data Retrieval Error Message. **[FR300.31] [M1]**
A1.3. System provides a Retry option.
A1.4. Tourist selects Retry.
A1.5. System attempts to retrieve the mission data again.
A1.6. Use case returns to Basic Flow Step 5.

A2: Pause Mission

Occurs at Basic Flow Step 28

A2.1. Tourist selects Pause Mission. **[FR300.23]**
A2.2. System displays the Pause Mission Confirmation Message. **[FR300.31] [M2]**
A2.3. Tourist confirms the pause action.
A2.4. System saves the current mission state and progress to the system database. **[FR300.24]**
A2.5. System records the mission pause date and time. **[FR300.24]**
A2.6. System changes the mission status to Paused. **[FR300.24]**
A2.7. System suspends GPS location tracking. **[FR300.24]**

A3: Resume Paused Mission

Occurs at Basic Flow Step 2

A3.1. Tourist selects a previously paused mission. **[FR300.25]**
A3.2. System retrieves the saved mission state and progress from the system database. **[FR300.25]**
A3.3. System changes the mission status from Paused to Active. **[FR300.26]**
A3.4. System resumes GPS location tracking. **[FR300.26]**
A3.5. Use case continues from Basic Flow Step 10.

A4: Abandon Mission

Occurs at Basic Flow Step 29

A4.1. Tourist selects Abandon Mission. **[FR300.27]**
A4.2. System displays the Abandon Mission Confirmation Message. **[FR300.31] [M3]**
A4.3. Tourist confirms the abandonment action.
A4.4. System changes the mission status to Abandoned. **[FR300.28]**
A4.5. System records the mission abandonment date and time. **[FR300.28]**
A4.6. System stops GPS location tracking. **[FR300.28]**
A4.7. System closes the active mission session. **[FR300.28]**
A4.8. System prevents the abandoned mission from being recorded as completed or triggering completion-based rewards. **[FR300.29]**
A4.9. System redirects the tourist to the home page.
A4.10. Use case ends.

A5: Tourist Outside Geofence

A5.1. System determines that the tourist is outside the required destination geofence.
A5.2. System continues retrieving the tourist's GPS location.
A5.3. System continues calculating the distance between the tourist and the hidden destination.
A5.4. System continues displaying updated Hot & Cold proximity feedback.
A5.5. Use case returns to Basic Flow Step 12.

A6: Insufficient GPS Accuracy

Occurs at Basic Flow Step 20

A6.1. System detects that the tourist's current GPS accuracy is insufficient to verify the destination visit.
A6.2. System displays the Poor GPS Accuracy Message. **[FR300.31] [M4]**
A6.3. System asks the tourist to wait or move to an open area with a stronger GPS signal.
A6.4. System retrieves updated latitude, longitude, and GPS accuracy information.
A6.5. Use case returns to Basic Flow Step 20.

A7: Destination Information Cannot Be Retrieved

Occurs at Basic Flow Step 24

A7.1. System fails to retrieve the verified destination information from the system database.
A7.2. System displays the Destination Information Retrieval Error Message. **[FR300.31] [M5]**
A7.3. System provides a Retry option.
A7.4. Tourist selects Retry.
A7.5. System attempts to retrieve the destination information again using the destination ID associated with the mission.
A7.6. After successfully retrieving the information, the use case returns to Basic Flow Step 25.

MESSAGE SECTION

M1: Mission Data Retrieval Error Message

"Unable to load the mission information. Please try again."

M2: Pause Mission Confirmation Message

"Are you sure you want to pause this mission? Your current progress will be saved."

M3: Abandon Mission Confirmation Message

"Are you sure you want to abandon this mission? This mission will not be counted as completed."

M4: Poor GPS Accuracy Message

"Your current GPS signal is not accurate enough to verify your location. Please wait or move to an open area and try again."

M5: Destination Information Retrieval Error Message

"Your destination visit has been verified, but the destination information could not be loaded. Please try again."

CONSTRAINT SECTION

C1: Display Mission Information

The mission summary must display:

- Mission status
- Mission category
- Estimated mission duration
- Start Mission option

The destination name, exact address, and exact geographical coordinates must remain hidden before successful destination verification.

C2: Display Active Mission Information

The active mission interface must display:

- View Mystery Clue option
- Current Hot & Cold proximity feedback
- Mission status
- Pause Mission option
- Abandon Mission option

The hidden destination identity, exact address, geographical coordinates, and exact map marker must remain concealed before successful destination verification.

C3: Hot & Cold Proximity Levels

The system must classify the tourist's proximity to the hidden destination according to predefined distance thresholds.

The proximity feedback may include:

- Freezing
- Cold
- Warm
- Hot
- Destination Reached

The distance thresholds may be configured according to destination type and GPS conditions.

The system should prevent frequent changes in proximity status caused by minor GPS fluctuations.

C4: Destination Concealment

Before successful destination verification, the system must not display:

- Destination name
- Exact address
- Exact latitude and longitude
- Exact destination marker
- Images that directly reveal the destination
- Information that clearly exposes the destination identity

C5: Destination Verification Rule

The destination visit may only be verified when:

1. The mission status is Active.
2. The tourist is within the configured geofence radius.
3. The GPS accuracy satisfies the required threshold.
4. The destination visit has not previously been verified.

C6: Display Revealed Destination Information

After successful destination verification, the system may display:

- Destination name
- Category
- Description
- Address
- Images
- Historical information
- Cultural significance
- Interesting facts
- Other available tourism information

C7: Pause and Abandon Availability

- The Pause Mission option must only be available when the mission status is Active.

• The Abandon Mission option may only be available when the mission status is Active or Paused. • A mission with the status Completed cannot be paused or abandoned.	
Post-Conditions	One of the following outcomes occurs: 1. The mission is completed, the destination visit is verified, and the destination information is displayed. 2. The mission is paused, and its current state and progress are saved. 3. The mission is abandoned and is not recorded as completed. 4. The latest mission state and progress successfully saved in the database remain available for later continuation. 5. GPS tracking is stopped or suspended when the mission is completed, paused, or abandoned.

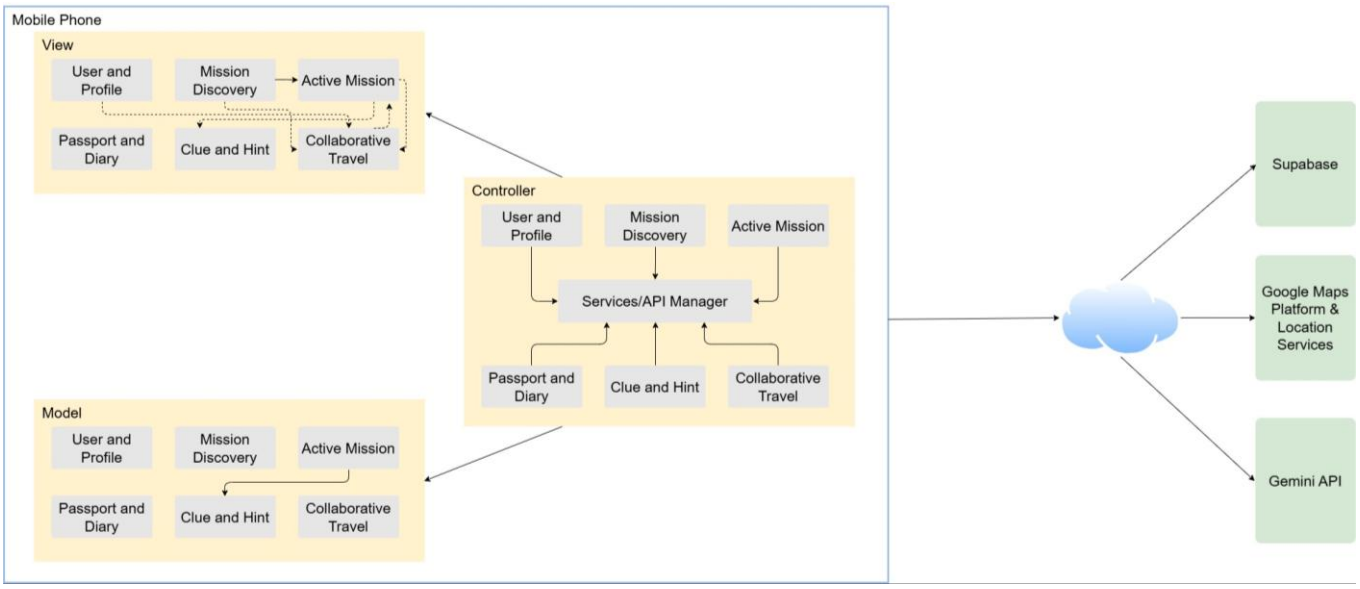
B3. Functional Requirements

USE CASE	FUNCTIONAL REQUIREMENT	
	REQ. ID	REQ. STATEMENT
UC300_ Manage Active Mission	FR300.1	The system shall display generated mystery missions and allow the tourist to select a mission.
	FR300.2	The system shall display the mission summary and allow the tourist to start the selected mission.
	FR300.3	The system shall create an active mission session, set its status to Active, and record the start date and time.
	FR300.4	The system shall retrieve the hidden destination ID and coordinates without revealing the destination identity.
	FR300.5	The system shall verify that a mystery clue is associated with the selected mission.
	FR300.6	The system shall save the latest mission status and progress to the system database.
	FR300.7	The system shall continuously retrieve the tourist's latitude, longitude, and GPS accuracy while the mission is active.
	FR300.8	The system shall display the active mission interface.
	FR300.9	The system shall calculate the distance between the tourist and the hidden destination.
	FR300.10	The system shall compare the current distance with the previously calculated distance.
	FR300.11	The system shall provide Hot & Cold proximity feedback.
	FR300.12	If the tourist selects View Mystery Clue, the system shall activate UC501_View Mystery Clue.
	FR300.13	The system shall detect when the tourist enters the destination geofence.
	FR300.14	The system shall validate the accuracy of the tourist's GPS location.
	FR300.15	The system shall verify the visit only when the tourist is within the geofence and GPS accuracy is acceptable.
	FR300.16	The system shall record the verified destination visit and related location information.
	FR300.17	The system shall update the mission progress after successful verification.
	FR300.18	The system shall retrieve the destination information after successful verification.
	FR300.19	The system shall display the verified destination information.
	FR300.20	The system shall prevent destination information from being revealed before successful verification.
	FR300.21	The system shall change the mission status to Completed after successful verification.
	FR300.22	The system shall stop GPS tracking after the mission is completed.
	FR300.23	The system shall allow the tourist to pause an active mission.
	FR300.24	The system shall save the mission state, record the pause time, set the status to Paused, and suspend GPS tracking.
	FR300.25	The system shall allow the tourist to resume a paused mission.
	FR300.26	The system shall set a resumed mission to Active and resume GPS tracking.
	FR300.27	The system shall allow the tourist to abandon an active or paused mission after confirmation.
	FR300.28	The system shall record an abandoned mission, stop GPS tracking, and close the mission session.
	FR300.29	The system shall prevent an abandoned mission from being recorded as completed or receiving completion rewards.

	FR300.30	The system shall retrieve the latest saved mission state after the application is reopened.
	FR300.31	The system shall display appropriate error and confirmation messages.

SECTION C: DESIGN STUDY

C1. Initial Design: Software Internal Structure



Justification

Mystery Malaysia uses a facade-mediated Model-View-Controller (MVC) architecture because the application contains several modules that must evolve in parallel while sharing external services. The Model layer stores typed domain data and deterministic state, such as mission status and progress. The View layer concentrates on Flutter presentation and user interaction, while Controllers validate actions, coordinate workflows, and translate service results into observable state. This separation keeps presentation decisions out of business logic and makes each responsibility easier to understand, test, and maintain.

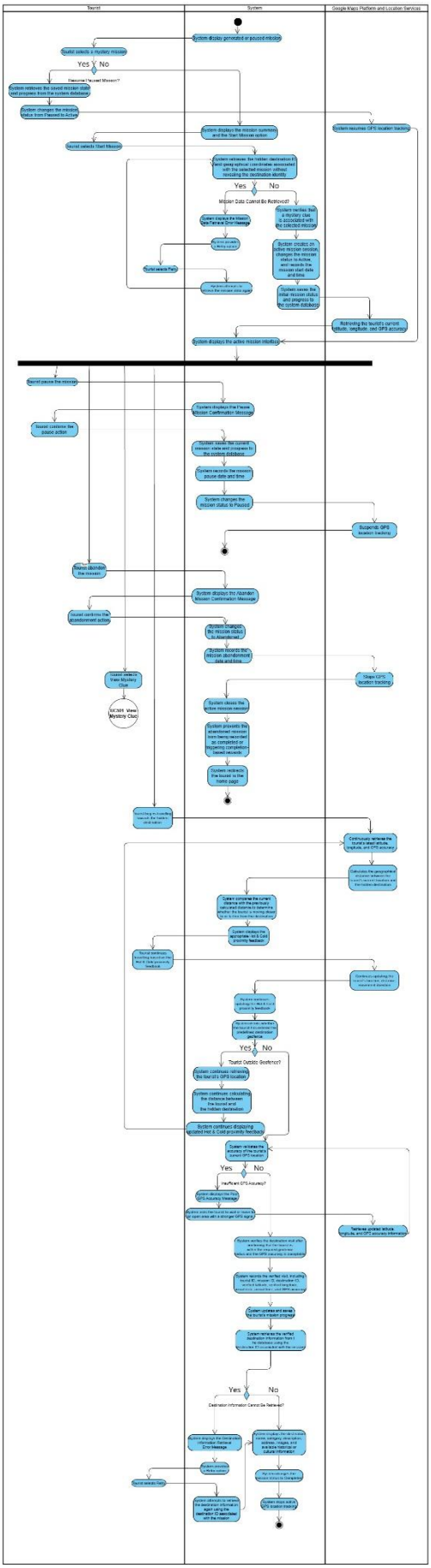
A module-specific facade interface is placed between each View and its Controller. The View depends only on a small contract, while the concrete facade delegates operations and forwards ChangeNotifier updates. Consequently, team members can work on User and Profile, Mission Discovery, Active Mission, Passport and Diary, Clue and Hint, or Collaborative Travel with fewer direct dependencies. Changes to Supabase, Google Maps, Gemini, or device APIs remain behind Services/API Managers instead of spreading provider-specific code across screens. The boundary also supports focused tests because Controllers can receive substitute services and Views do not construct infrastructure objects.

The Active Mission module demonstrates this structure clearly. ActiveMissionView renders mission progress and forwards start, pause, resume, abandon, and location actions through IActiveMissionFacade. ActiveMissionController owns the asynchronous mission workflow, MissionSession represents status and progress, and the location and Supabase services handle device GPS and protected backend requests. Sensitive destination coordinates and authoritative arrival rules therefore remain outside the View. This design was selected over placing all logic in widgets because it reduces coupling, gives the team stable collaboration boundaries, and allows the managed backend to remain the source of truth for verification, completion, and rewards.

C2. Detailed Design

C2(a) Activity Diagram

https://drive.google.com/file/d/193BBhDwbRQNadP3twyi1yd8iBJQtc9Rp/view?usp=sharing


Tourist
System
Google Maps Platform and Location Services
System displays generated or paused mission
Tourist selects a Mystery mission
Yes
No
Resume Paused Mission?
System retrieves the saved mission data and progress from the system database
System changes the mission status from Paused to Active
System displays the mission summary and the Start Mission option
System resumes GPS location tracking
Tourist selects Start Mission
System retrieves the hidden destination ID and geographical coordinates associated with the selected mission without revealing the destination identity
Yes
No
Mission Data Cannot Be Retrieved?
System displays the Mission Data Retrieval Error Message
System verifies that the tourist is associated with the selected mission
System creates an active mission session, changes the mission status to Active, and records the mission start date and time
System saves the active mission status and progress to the system database
Retrieving the tourist's current latitude, longitude, and GPS accuracy
System displays the active mission interface
Tourist pause the mission
System displays the Pause Mission Confirmation Message
Tourist confirms the pause action
System saves the current session's state and progress to the system database
System records the mission pause date and time
System changes the mission status to Paused
Suspends GPS location tracking
Tourist abandon the mission
System displays the Abandon Mission Confirmation Message
Tourist confirms the abandonment action
System changes the mission status to Abandoned
System records the mission abandonment date and time
Stops GPS location tracking
Tourist selects View Mystery Clue
UC301 View Mystery Clue
System closes the active mission session
System prevents the system from resuming tasks having recorded as completed or triggering completion-based rewards
System redirects the tourist to the home page
Continuously retrieves the tourist's latest latitude, longitude, and GPS accuracy
Calculates the geographical distance between the tourist's current location and the hidden destination
System compares the current distance with the previously calculated distance to determine whether the tourist is moving closer to or further away from the destination
System displays the appropriate Cold/Hot proximity feedback
Tourist continues travelling but not in the Hot & Cold proximity feedback
Continuously updating the tourist's location, distance, and direction
Yes
No
Tourist Outside Geofence?
System continues retrieving the tourist's GPS location
System continues calculating the distance between the tourist and the hidden destination
System continues displaying updated Hot & Cold proximity feedback
System retrieves the verified destination ID using the current GPS location
Yes
No
Insufficient GPS Accuracy?
System displays the GPS Accuracy Message
Retrieves updated latitude, longitude, and GPS accuracy information
System verifies the destination and pre-validates that the tourist is within the required geofence radius and the GPS accuracy is acceptable
System records the verified visit, including tourist ID, mission ID, destination ID, verified latitude, and verified longitude
System updates and saves the tourist's mission progress
System retrieves the verified destination ID using the tourist's verified location and matches it in accordance with the mission
Yes
No
Destination Information Cannot Be Retrieved?
System displays the Destination Information Retrieval Error Message
System retrieves the destination name, category, description, address, images, and available historical or cultural information
Tourist selects Retry
Tourist selects Exit
System attempts to retrieve the destination information again using the destination ID associated with the mission
System closes active GPS location tracking

C3. Software UI / GUI

<insert the GUIs of your module/application segment>

- *<snapshot from the system used in the presentation>,* ***AND***
- *<insert link to the mock-up in Figma – do not modify the mock-up in Figma>*

SECTION D: IMPLEMENTATION / CODING

D1. Code Segment 1 - Starting a saved mission safely

Source: lib/controller/active_mission/active_mission_controller.dart - start()

```
Future<bool> start() async {
  if (session.status != MissionStatus.ready || sessionId == null) {
    return false;
  }
  isUpdatingStatus = true;
  arrivalError = null;
  notifyListeners();
  try {
    final launch = await _dataService.startSelectedMission(sessionId!);
    currentClue = launch.clue;
    session = session.copyWith(
      currentStage: launch.currentStage,
      status: launch.status,
    );
    return true;
  } on ApiError catch (error) {
    arrivalError = error.message;
    return false;
  } finally {
    isUpdatingStatus = false;
    notifyListeners();
  }
}
```

Importance: This method is the controlled entry point from a saved Ready mission into live navigation. It rejects an invalid local state or a missing session identifier before any request is sent, then asks the backend to perform the actual start transition. The screen changes to Active only after the returned MissionLaunch is accepted, so the client does not invent an authoritative state.

Learning points: I learned that an asynchronous state transition should expose loading, success, and failure states explicitly. notifyListeners() lets the View disable repeated actions and show progress, while the finally block guarantees that the loading flag is cleared even when the network request fails.

Uniqueness: The response updates both the current stage and clue reference in one operation. This keeps the UI consistent with the server's selected mission data while preserving the facade-mediated MVC boundary.

Challenges: The main challenge was avoiding an optimistic local transition that could disagree with Supabase. The method therefore waits for the service result and converts ApiError into a user-safe message instead of leaving a partially started mission on screen.

D2. Code Segment 2 - Pause, resume, and abandon lifecycle controls

Source: lib/controller/active_mission/active_mission_controller.dart - togglePause() and abandon()

```
Future<bool> togglePause() async {
  final next = session.status == MissionStatus.paused
      ? MissionStatus.active
      : MissionStatus.paused;
  isUpdatingStatus = true;
  arrivalError = null;
  notifyListeners();
  try {
    final status = sessionId == null
        ? next
        : await _dataService.updateMissionStatus(
            sessionId!,
            next == MissionStatus.paused ? 'pause' : 'resume',
          );
    session = session.copyWith(status: status);
    return true;
  } on ApiError catch (error) {
    arrivalError = error.message;
    return false;
  } finally {
    isUpdatingStatus = false;
    notifyListeners();
  }
}

Future<bool> abandon() async {
  isUpdatingStatus = true;
  arrivalError = null;
  notifyListeners();
  try {
    final status = sessionId == null
        ? MissionStatus.abandoned
        : await _dataService.updateMissionStatus(sessionId!, 'abandon');
    session = session.copyWith(status: status);
    return true;
  } on ApiError catch (error) {
    arrivalError = error.message;
    return false;
  } finally {
    isUpdatingStatus = false;
    notifyListeners();
  }
}
```

Importance: Pause, resume, and abandon are critical mission lifecycle operations because they determine whether location tracking should continue and whether progress remains eligible for completion. The methods send a small action command and update the local MissionSession only with the status returned by the backend. The View separately requests confirmation before destructive abandonment.

Learning points: I learned to model lifecycle changes as explicit states rather than several unrelated Boolean flags. Deriving next from the current MissionStatus reduces branching, and copyWith creates a new consistent session value without mutating individual fields.

Uniqueness: The same Controller supports a demonstration session and a persisted session. Demo mode applies the next status locally, whereas a real session delegates to the protected mission-control operation. Both paths expose the same result to the View.

Challenges: Network delay and repeated taps could otherwise create contradictory transitions. The shared isUpdatingStatus flag allows the interface to disable actions, while try/finally restores a stable UI state and ApiError preserves a clear failure message.

D3. Code Segment 3 - Guarded periodic GPS sampling

Source: lib/view/active_mission/active_mission_view.dart - initState(), dispose(), and _trackLocation()

```
@override
void initState() {
  super.initState();
  facade = widget.facade;
  WidgetsBinding.instance.addPostFrameCallback((_) => _trackLocation());
  if (facade.sessionId != null) {
    locationTimer = Timer.periodic(const Duration(seconds: 15), (_) async {
      if (facade.session.status == MissionStatus.active &&
          !facade.isLocating &&
          !facade.isVerifying &&
          !arrivalSheetOpen) {
        await _trackLocation();
      }
    });
  }
}

@override
void dispose() {
  locationTimer?.cancel();
  facade.dispose();
  super.dispose();
}

Future<void> _trackLocation() async {
  if (!mounted ||
      arrivalSheetOpen ||
      facade.isLocating ||
      facade.isVerifying ||
      facade.session.status != MissionStatus.active) {
    return;
  }
  final result = await facade.refreshLocation();
  if (!mounted || result == null) return;
  arrivalSheetOpen = true;
  await _showDestinationGuide(result);
  arrivalSheetOpen = false;
}
```

Importance: This code drives foreground navigation by sampling location every 15 seconds while a persisted mission is Active. It performs one immediate post-frame check, prevents overlapping location or verification requests, and opens the destination guide only when the Controller returns a verified result.

Learning points: I learned that location intelligence is not only a calculation problem; it is also a scheduling and widget-lifecycle problem. A controlled sampling interval reduces unnecessary GPS, network, and battery use compared with starting an unrestricted stream. The timer must also be cancelled during dispose().

Uniqueness: The guards combine mission state, asynchronous operation flags, widget mounting, and modal state. This prevents a second verification or destination sheet from appearing while the first result is still being processed.

Challenges: The difficult part was asynchronous re-entry: a timer can fire while the previous call is awaiting GPS or while the user leaves the screen. The mounted and busy-state checks make those race conditions explicit and keep the View responsive.

D4. Code Segment 4 - Proximity intelligence and automatic arrival trigger

Source: lib/controller/active_mission/active_mission_controller.dart - refreshLocation()

```
Future<ArrivalResult?> refreshLocation() async {
  if (session.status != MissionStatus.active) return null;
  ArrivalResult? automaticArrival;
  isLocating = true;
  locationError = null;
  notifyListeners();

  try {
    final location = await _locationService.getCurrentLocation();
    currentLocation = location;
    if (sessionId != null && session.status == MissionStatus.active) {
      await _syncMission();
      if (session.status != MissionStatus.active) return null;
      proximityLevel = await _dataService.missionProximity(
        sessionId!,
        location,
      );
      if (proximityLevel == 'reached' &&
          _verifiedStage != session.currentStage) {
        automaticArrival = await confirmArrival();
      }
    } else if (sessionId == null) {
      session = session.copyWith(
        distanceMetres: _locationService.distanceMetres(
          location,
          mysteryZone,
        ),
      );
    }
  } on LocationServiceException catch (error) {
    locationError = error.message;
  } on ApiError catch (error) {
    locationError = error.message;
  } finally {
    isLocating = false;
    notifyListeners();
  }
  return automaticArrival;
}
```

> **Importance:** This is the core location-intelligence loop. It obtains the tourist's latest GPS reading, synchronises the mission in case another operation changed its status, and sends the location to the backend for a Hot and Cold classification. A server response of reached triggers arrival verification automatically.

Learning points: I learned to keep secret and authoritative rules away from the client. The Controller knows the tourist's location and the returned proximity label, but the real hidden coordinates and configurable geofence remain on the server. This protects the mystery and prevents the View from becoming the source of truth.

Uniqueness: The _verifiedStage guard ensures that periodic checks verify a reached stage only once. The method also supports the non-persisted demonstration path through the same public operation, using a local distance only when no real session identifier exists.

Challenges: GPS permission, weak accuracy, network failure, and a mission completed elsewhere can all occur during one refresh. Separate exception handling produces an understandable location error, while _syncMission() prevents stale screen state from starting another verification.

D5. Code Segment 5 - Verified arrival and mission progression

Source: lib/controller/active_mission/active_mission_controller.dart - confirmArrival()

```
Future<ArrivalResult?> confirmArrival() async {
  if (session.status != MissionStatus.active) return null;
  final location = currentLocation;
  if (location == null) {
    arrivalError = 'Refresh your GPS location before confirming arrival.';
    notifyListeners();
    return null;
  }

  isVerifying = true;
  arrivalError = null;
  notifyListeners();
  try {
    final result = sessionId == null
        ? _demoArrival()
        : await _dataService.confirmArrival(
            sessionId: sessionId!,
            location: location,
          );
    if (!result.arrived) {
      arrivalError = result.message;
      return null;
    }
    _verifiedStage = session.currentStage;
    if (result.nextClue != null) currentClue = result.nextClue!;
    session = session.copyWith(
      currentStage: result.nextStage ?? session.currentStage,
      status: result.missionCompleted ? MissionStatus.completed : null,
    );
    return result;
  } on ApiError catch (error) {
    arrivalError = error.message;
    return null;
  } finally {
    isVerifying = false;
    notifyListeners();
  }
}
```

Importance: Arrival verification is the trust boundary between navigation and progression. The method requires an Active mission and a current GPS reading, then delegates the authoritative geofence and GPS-accuracy decision to the backend. Only an arrived result can reveal the destination guide or advance the mission.

Learning points: I learned to treat the service response as one coherent transition result rather than making separate requests for verification, next stage, next clue, and completion. Applying those returned values together reduces temporary inconsistencies in the View.

Uniqueness: The same ArrivalResult supports both a multi-stage continuation and final mission completion. It can carry the revealed guide, the next clue, and the next stage, while MissionSession.copyWith preserves unchanged fields. Marking _verifiedStage also cooperates with the periodic tracking guard in Segment 4.

Challenges: A reached proximity is not sufficient by itself because GPS accuracy may still be poor and repeated timer ticks may arrive during verification. The dedicated isVerifying flag, explicit failure message, and server-side result keep the workflow safe without revealing protected destination data.

SECTION E: SELF-REFLECTION

E1. What is the most significant lesson you learned from this exercise? Explain why it is meaningful and describe a specific experience from the exercise that led to this learning

The most significant lesson I learned was that reliable software depends on clear ownership of state and responsibility, not only on making a screen work. This became meaningful while implementing automatic arrival detection for the active_mission module. The screen checks location every 15 seconds, so a simple reached result could have submitted the same arrival repeatedly. I added busy-state guards in the View and a verified-stage guard in the Controller, while keeping the final geofence and GPS-accuracy decision on the backend. This experience showed me why the client should coordinate the interaction but should not be the authority for sensitive or reward-related transitions. It also connected several course concepts in one feature: MVC separation, asynchronous programming, error handling, testability, and secure data boundaries. The result was not merely a Hot and Cold interface; it was a controlled workflow that could tolerate repeated timer events, weak GPS, network failure, and a mission state that had changed elsewhere.

E2. Has this exercise changed your perception of the software development process (from ideation to implementation)? Explain how your perception has changed and provide examples from the exercise to support your answer

This exercise changed my view of software development from a mostly linear sequence into an iterative process where requirements, design, implementation, and testing continuously influence one another. At first, UC300 appeared to be a straightforward flow: start a mission, navigate, and confirm arrival. When I translated the use case and FR300 requirements into code, each statement created design questions. Pause and resume required explicit states; destination concealment required the coordinates to remain outside the View; automatic arrival required concurrency guards; and poor GPS accuracy required a recoverable error path. These discoveries led me back to the architecture and API contract several times. I also saw the value of implementing a small vertical path from the Flutter View through the facade and Controller to Supabase, then testing the state returned to the UI. The process was therefore less about completing phases once and more about validating assumptions early and refining the design as evidence appeared.

E3. How has this exercise influenced your understanding of collaborative software development? Discuss both the benefits and challenges you experienced while working in a team

The exercise strengthened my understanding that collaborative development depends on stable boundaries and communication as much as individual coding skill. A major benefit was that our modules could progress in parallel. I concentrated on UC300 Manage Active Mission, while UC501 View Mystery Clue belonged to another team member. The facade interfaces and typed models gave us clear connection points: active_mission could pass the current clue and session context without taking ownership of hint or clue logic. Shared architecture rules also made it easier to review where code should be placed. The challenge was that one change to a status value, payload field, or navigation contract could affect several modules even when the source files were separate. We needed to agree on terms such as Ready, Active, Paused, Completed, and Abandoned and communicate changes before integration. I learned that short, specific discussions about interfaces and acceptance criteria prevent much larger merge and debugging problems later.

E4. Reflect on your performance during this exercise. Identify your key strengths and areas for improvement, and explain how each affected your team's progress or project outcomes

My main strengths were translating detailed requirements into explicit state transitions and handling asynchronous edge cases carefully. I traced the active mission lifecycle, kept the hidden destination outside the UI, separated location errors from arrival errors, and ensured that periodic checks could not overlap. These decisions helped the team by producing a predictable module boundary and reducing the risk that mission progress or completion would be updated incorrectly. I also used dependency injection in the Controller test to verify that a reached GPS check only confirms arrival once for the current stage. My main area for improvement is testing earlier on multiple physical devices and under more realistic location conditions. Emulator and service-level tests cannot fully reproduce denied permissions, poor accuracy near buildings, backgrounding, or intermittent connectivity. I could also have documented the expected API responses and error codes earlier. Doing so would have reduced late clarification and made integration with the shared backend smoother.

E5. If you were to repeat this exercise, what specific actions would you take differently? Explain how these changes would improve your learning experience or the project outcome.

If I repeated the exercise, I would begin by agreeing on a concise contract for mission statuses, facade operations, Supabase payloads, and user-visible errors before building the complete interface. I would create a small traceability checklist that links each FR300 requirement to one implementation point and at least one acceptance check. Next, I would deliver the module in short vertical slices: start a saved mission, obtain one location, display one server proximity result, verify one arrival, and only then add pause, resume, abandonment, and presentation polish. I would also schedule device tests earlier for permission denial, weak GPS accuracy, timer overlap, app backgrounding, and network interruption. Finally, I would use smaller integration checkpoints with the Clue and Hint and Mission Discovery owners instead of waiting for a larger combined test. These actions would reveal contract mismatches sooner, reduce rework, and leave more time for usability and reliability improvements.